Artificial Vision

Prof. Mario Vento – Prof. Antonio Greco

Anno Accademico 2023/2024

# PAR Contest Project Work

Gruppo 04


| | | |
|---|---|---|
| Lombardi Mariapia | m.lombardi59@studenti.unisa.it | 0622701957 |
| Mignone Lorenzo | l.mignone@studenti.unisa.it | 0622701866 |
| Mucciolo Francesco | f.mucciolo4@studenti.unisa.it | 0622702088 |
| Santonicola Carlo | c.santonicola7@studenti.unisa.it | 0622702027 |

INDICE

INTRODUZIONE ..... 3
1. DETECTOR ..... 3
1.1. YOLO 8 ..... 3
2. TRACKER ..... 5
2.1. DEEP SORT ..... 5
2.1.1 SORT ..... 5
2.1.2 PRINCIPIO DI FUNZIONAMENTO ..... 6
2.1.3 TRACK MANAGEMENT MODULE ..... 7
2.1.4 PERFORMANCE EVALUATION ..... 7
2.1.5. FINE TUNING DEL DETECTOR E TRACKER ..... 8
3. PAR ..... 8
3.1. DATASET ..... 8
3.2. NETWORK ARCHITECTURE ..... 10
3.3. TRAINING ..... 13
3.3.1 PRE-PROCESSING E DATA AUGMENTATION ..... 13
3.3.2 CUSTOM SAMPLER ..... 14
3.3.3 PROCEDURA DI ADDESTRAMENTO ..... 14
3.3.4. CUSTOM LOSS ..... 15
3.3.5. EVALUATION PROTOCOL ..... 16
3.4. EXPERIMENTAL RESULTS ..... 17
4. APPLICATION ..... 17
4.1. UPDATE ROI ..... 17
4.2. UPDATE PAR ..... 18
4.3. PRINT_SCENE ..... 20
Bibliografia ..... 21
Indice delle Figure ..... 22

# INTRODUZIONE

Il riconoscimento degli attributi dei pedoni "PAR" assume un ruolo di rilievo nell'ambito dell'intelligenza artificiale, trovando applicazioni diverse come sorveglianza video, sicurezza urbana e interazioni sociali, come nel caso dei robot sociali. È un problema ampiamente studiato, ma resta ancora irrisolto a causa di sfide quali scarsa illuminazione, pose non ideali, occlusioni e bassa risoluzione delle immagini per soggetti distanti dalla telecamera.

Nel presente documento, vengono analizzati in dettaglio i componenti del sistema:

- ***detector***: responsabile della individuazione delle persone nei frame
- **t*racker:*** incaricato del tracciamento delle persone assegnando loro un id univoco
- ***CNN multitask***: una rete neurale utilizzata per la classificazione degli attributi.

Per ciascuna persona tracciata, vengono registrati *id, genere, colore della parte superiore e inferiore dell'abbigliamento, presenza di borsa e cappello*. Inoltre, vengono fornite informazioni spaziali e temporali aggiuntive, inclusi il *tempo di permanenza* e il *numero di passaggi* all'interno delle due regioni d'interesse della scena, configurabili durante l'avvio dell'applicazione. I risultati sono poi stampati in un file di testo.

# 1. DETECTOR

Nel sistema, il detector riveste un ruolo di primaria importanza poiché permette la rilevazione e la localizzazione delle persone all'interno del frame dell'immagine. La scelta è stata orientata verso l'utilizzo di un'architettura *single-stage*, che garantisce prestazioni in tempo reale superiori rispetto alla controparte *two-staged*. Questo tipo di architettura consente la classificazione e la localizzazione di un oggetto contemporaneamente, mediante l'impiego di box predefiniti con dimensioni e rapporti di aspetto diversi.

## 1.1. YOLO 8

La versione più recente del modello offerto da "Ultralytics" per il rilevamento degli oggetti è *Yolo8 (You Only Look Once)*, noto per ottimizzare il trade-off tra velocità di inferenza e precisione molto utile in contesti realtime. Yolo8 presenta miglioramenti significativi rispetto alle versioni precedenti, grazie alle modifiche apportate alla backbone *CSPDarknet*, che migliora l'estrazione delle features, e all'impiego dell'*anchor free-detection* sebbene si tratti di un modello *single-stage*. Quest'ultimo metodo prevede la predizione diretta del centro di un oggetto sfruttando metodi alternativi, come ad esempio le mappe di calore, invece di predire l'offset da una *anchor-box* nota. Ciò garantisce maggiore efficienza computazionale. Le diverse versioni del modello Yolo8 includono *nano, medium, large* ed *x*, ognuna con performance e dimensioni variabili. La scelta della versione ***small*** è motivata dalle ottime predizioni ottenute senza compromettere la velocità di elaborazione. La versione *small* conta circa 11.2 milioni di parametri ed è addestrata sul dataset *COCO* utilizzando immagini di dimensione 640x640, con un mAP del 44.9%. (ultralytics, s.d.).

Di seguito sono elencate le metriche considerate nella scelta del detector:

- **Intersection over Union (IoU)*:*** è definita come il rapporto tra l'area di intersezione e l'area di unione tra il bounding box predetto e quello di ground truth

$$IoU = \frac{Area\ di\ Intersezione}{Area\ di\ Unione}$$

- **Precision** e **Recall**: Precision rappresenta la percentuale di veri positivi tra tutte le previsioni positive, mentre Recall rappresenta la percentuale di veri positivi tra tutti i positivi effettivi.

$$Precision = \frac{Veri\ Positivi}{Veri\ Positivi\ +\ Falsi\ Positivi}$$

$$Recall = \frac{Veri\ Positivi}{Veri\ Positivi\ +\ Falsi\ Negativi}$$

- **Precisione Media (AP)**: Calcola l'area sotto la curva di precision-recall fornendo un unico valore che racchiude entrambe le metriche.
- **Precisione Media (mAP)**: Calcola i valori medi di AP su più classi di oggetti.
- **FPS**: Indica i frame per secondo e viene utilizzato per valutare la velocità di elaborazione di un frame del video.

```
Ultralytics YOLOv8.1.11 🚀 Python-3.8.18 torch-2.2.0+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Laptop GPU, 3904MiB)
YOLOv8s summary (fused): 168 layers, 11156544 parameters, 0 gradients, 28.6 GFLOPs
val: Scanning /home/lorx/Documents/GitHub/artificial-vision/datasets/coco128/labels/train2017.cache... 126 images
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|          | 8/8 [0
                   all        128        929      0.692      0.604      0.688      0.478
                person        128        254      0.853      0.709       0.83      0.602
               bicycle        128          6      0.698      0.397      0.523      0.286
                   car        128         46      0.796      0.255      0.456      0.269
            motorcycle        128          5      0.646        0.8      0.838      0.492
              airplane        128          6      0.706          1      0.879      0.585
                   bus        128          7      0.642      0.714      0.776      0.653
                 train        128          3      0.467      0.667      0.806      0.392
                 truck        128         12      0.743        0.5      0.517      0.356
                  boat        128          6      0.307      0.167      0.357       0.17
         traffic light        128         14          1      0.321      0.548      0.286
             stop sign        128          2          1      0.909      0.995      0.921
                 bench        128          9      0.664      0.556      0.553       0.26
                  bird        128         16      0.835      0.875      0.932      0.662
                   cat        128          4      0.742          1      0.945      0.672
                   dog        128          9      0.799      0.886      0.862      0.627
```

*Figura 1 Analisi delle prestazioni Yolo 8*

```
Speed: 3.1ms preprocess, 24.3ms inference, 0.0ms loss, 2.9ms postprocess per image
```

*Figura 2 Analisi velocità di processamento Yolo 8*

I seguenti risultati sono stati ottenuti attraverso un'analisi preliminare delle prestazioni su immagini di testing del dataset *coco128* utilizzando il comando:

yolo detect val model=yolov8s.pt data=coco128.yaml

In particolare, focalizzandosi sulla classe d'interesse ***person***, le metriche *mAP50* e *mAP50-95* rappresentano rispettivamente la precisione media calcolata con una soglia di *IoU* di 0.5 e con soglie variabili comprese tra 0.5 e 0.95. È stata selezionata una soglia **IoU** di **0.7** per ridurre il

numero di duplicati senza compromettere eccessivamente la capacità di rilevamento. Inoltre, le dimensioni scelte di **larghezza** e **altezza** del frame per effettuare la detection sono rispettivamente **960** e **544**, per mantenere il rapporto d'aspetto del video originale che è pari a **16:9**. Ulteriori parametri settati opportunamente includono il numero di **frame per secondo (FPS)** e la **confidence**, che saranno trattati più dettagliatamente nell'analisi dedicata al tracker, essendo stati soggetti a un fine tuning specifico.

# 2. TRACKER

L'associazione di un *ID* ad ogni persona rilevata nel frame dal detector è resa possibile dal tracker effettuando un'analisi temporale. Tale strumento permette la re-identificazione di una persona in caso di occlusioni, e consente di effettuare misurazioni su più frame degli attributi della persona in esame.

## 2.1. DEEP SORT

Il tracker utilizzato per il tracciamento di pedoni in movimento a passo d'uomo è il **DEEP SORT**, una versione avanzata del SORT. Il deep sort è un tipo di tracker a **singola ipotesi**, il che significa che viene utilizzato per tracciare una sola ipotesi riguardante la posizione e il movimento di un oggetto nel tempo. La scelta di utilizzare questo approccio è giustificata dall'intenzione di sfruttare i vantaggi delle predizioni del **filtro di Kalman**, integrando **l'informazione sull'apparenza** per mantenere traccia dei track anche in situazioni di affollamento. Per agevolare la comprensione del principio di funzionamento, si procede prima con la spiegazione del SORT.

### 2.1.1 SORT

Calcola la propagazione del bounding box del frame corrente al frame successivo attraverso l'applicazione di un **modello lineare a velocità costante**. Lo stato del target viene modellato considerando il suo centro, la scala e il rapporto d'aspetto. Successivamente, il filtro di Kalman viene applicato per la predizione del nuovo bounding box del target tramite il *motion model*, il quale fornisce uno stato predetto. In seguito all'operazione di *data association*, il seguente stato viene ulteriormente corretto con la rilevazione del nuovo bounding box del target. L' associazione bounding box-target consiste nella creazione di una matrice di costo basata sull'*IoU distance* di dimensione *NxM*, dove *N* è il numero di bounding box rilevati e *M* il numero di bounding box predetti. Questa matrice di costo viene risolta efficientemente attraverso *l'algoritmo Ungherese*. In caso di mancata associazione con il target, lo stato viene solo predetto ma non corretto.

Inizialmente, il tracciamento di un nuovo oggetto prevede l'inizializzazione con il bounding box appena rilevato e con una velocità iniziale nulla, poiché è una misura ancora non

osservata. Pertanto, la componente relativa alla velocità viene inizializzata con valori elevati di covarianza per rappresentare l'incertezza associata alla misura.

(University, arXiv, s.d.) (Ricardo Pereira, s.d.)

### 2.1.2 PRINCIPIO DI FUNZIONAMENTO

Il deep sort opera in modo analogo a quanto descritto nel paragrafo precedente, integrando l'informazione sull'apparenza del target estratta tramite CNN addestrate su dataset di person re-identification. Questa combinazione aggiuntiva assicura una **riduzione degli id-switch**, ai quali il SORT è particolarmente suscettibile.

Nel filtro di Kalman, lo stato è definito in uno spazio a otto dimensioni che include il centro del bounding box, il rapporto d'aspetto, l'altezza e la velocità espresse in coordinate frame. L'operazione di *data association* tra lo stato predetto dal filtro di Kalman e le nuove misurazioni viene risolta efficientemente con *l'algoritmo Ungherese*. Il problema viene formalizzato attraverso due metriche:

- **il quadrato della distanza di Mahalanobis**: Questa metrica tiene conto dell'informazione relativa al movimento calcolando la distanza tra gli stati predetti dal filtro di Kalman e le nuove misurazioni. La formula è la seguente:

$$d^{(1)}(i,\, j) \;=\; \left(d_j \;-\; y_i\right)^T S_i^{-1}\left(d_j \;-\; y_i\right)$$

  dove $yi$ rappresenta la proiezione dell' i-esimo track nello spazio di misura $(yi, Si)$ e $dj$ rappresenta il j-esimo bounding box rilevato. Dunque, calcola quante deviazioni standard delle nuove detection si discostano dalla media del track. L'ammissibilità dell'associazione è soggetta ad una *threshold*.

- **la distanza coseno**: Questa metrica considera l'informazione legata all'apparenza calcolando per ogni nuovo bounding box dj il suo descrittore $rj$. In particolare, si prendono in esame gli ultimi $N$ descrittori di ciascun track, calcolando la distanza minima tra l'i-esimo track e la proiezione del j-esimo bounding box rilevato nello spazio di apparenza. La formula è la seguente:

$$d^{(2)}(i,\; j) \;=\; \min\left\{1 \;-\; r_j^T r_k^{(i)} \;\parallel\; r_k^{(i)} \;\in\; R_i\right\}$$

$$R_k = \left\{r_k{}^{(i)}\right\}_{k=1}^{L_k}$$

  in cui $L_k$ sono gli ultimi descrittori esaminati per il track $k$. Anche in questo caso, l'ammissibilità dell'associazione è soggetta ad una *threshold*.

In conclusione, le due metriche vengono combinate attraverso una combinazione lineare pesata espressa come:

$$C_{i,j} = \lambda d^{(1)}(i, j) + (1 - \lambda) d^{(2)}(i, j)$$

L'associazione è confermata se entrambe le metriche soddisfano la condizione di ammissibilità.

Un caso tipico che porta ad associazioni errate è quando un track è soggetto ad occlusione, causando un aumento dell'incertezza nella predizione del *filtro di Kalman* relativa alla localizzazione del track nel frame. Di conseguenza, la distribuzione di probabilità si distribuisce, rendendo il track meno definito e quindi *la distanza di Mahalanobis* tenderà a preferire misurazioni con grandi deviazioni standard poiché minimizzano la distanza. Questo comportamento è controproducente in quanto genera associazioni errate. Una soluzione a questo problema è **prioritizzare l'associazione dei track più frequenti**, cioè quelli con *max age* minore.

(University, arXiv, s.d.)

### 2.1.3 TRACK MANAGEMENT MODULE

Il modulo in questione si occupa della gestione dei track. Il tracciamento di un nuovo oggetto avviene dopo aver rilevato la sua presenza per un numero di frame pari a *N*, rendendolo così ***confermato***. Finché il numero di frame è inferiore a *N*, l'oggetto si trova nello stato di ***tentativo***, che successivamente può essere *confermato* o *rimosso*. Durante questa fase viene assegnato un *ID* al track. Per ciascun track *k*, viene conteggiato il numero di frame dall'ultima corretta associazione, $a_k$. Il contatore viene incrementato durante la fase di predizione del filtro di Kalman e resettato ad ogni nuova associazione. Se il contatore di un determinato track supera la soglia *Max Age*, si presume che il track sia uscito dalla scena e quindi viene ***rimosso***.

### 2.1.4 PERFORMANCE EVALUATION

| | | MOTA ↑ | MOTP ↑ | MT ↑ | ML ↓ | ID ↓ | FM ↓ | FP ↓ | FN ↓ |
|---|---|---|---|---|---|---|---|---|---|
| KDNT [16]* | BATCH | 68.2 | 79.4 | 41.0% | 19.0% | 933 | 1093 | 11479 | 45605 |
| LMP_p [17]* | BATCH | **71.0** | **80.2** | **46.9%** | 21.9% | 434 | **587** | 7880 | **44564** |
| MCMOT_HDM [18] | BATCH | 62.4 | 78.3 | 31.5% | 24.2% | 1394 | 1318 | 9855 | 57257 |
| NOMTwSDP16 [19] | BATCH | 62.2 | 79.6 | 32.5% | 31.1% | **406** | 642 | **5119** | 63352 |
| EAMTT [20] | **ONLINE** | 52.5 | 78.8 | 19.0% | 34.9% | 910 | **1321** | **4407** | 81223 |
| POI [16]* | **ONLINE** | **66.1** | 79.5 | **34.0%** | 20.8% | 805 | 3093 | 5061 | **55914** |
| SORT [12]* | **ONLINE** | 59.8 | **79.6** | 25.4% | 22.7% | 1423 | 1835 | 8698 | 63245 |
| Deep SORT (Ours)* | **ONLINE** | 61.4 | 79.1 | 32.8% | **18.2%** | **781** | 2008 | 12852 | 56668 |

*Figura 3 Analisi Performance Deep Sort*

Nell'immagine sono riportati i benchmark ottenuti nel dataset *MOT16*.

(University, s.d.))

### 2.1.5. FINE TUNING DEL DETECTOR E TRACKER

Dopo la selezione del tracker e del detector, sono state condotte una serie di prove su diversi video al fine di ottimizzare i parametri. La decisione di utilizzare un **frame rate di 5 fps** è stata presa per bilanciare la complessità computazionale e la capacità di tracciare improvvisi cambi di direzione. Una **confidence** impostata al **0.6** consente di rilevare il maggior numero possibile di persone aumentando però il numero di falsi positivi. Tuttavia, quest'ultimi sono stati scartati mediante il fine tuning del tracker: infatti, per avviare il tracciamento di un nuovo oggetto, esso deve essere visibile in almeno **7 frame**, regolato da **N_INIT** che corrispondono a circa 1,5 secondi. Il valore di **MAX_AGE**, impostato a **30**, equivale a una mancata rilevazione di 6 secondi del track, comportandone la rimozione in quanto si assume non siano più presenti nella scena. Le threshold che controllano le metriche di movimento e dell'apparenza sono **MAX_IOU_DISTANCE** e **MAX_COSINE_DISTANCE**, i cui valori sono rispettivamente **0.7** e **0.3**. Si è optato per un valore tollerante di 0.7 in quanto procedendo a 5 fps si possono calcolare distanze più o meno grandi tra la predizione del filtro di Kalman e la misurazione. Contrariamente, per le features sull'apparenza si è scelto di essere quanto più stringenti possibile essendo specifiche per la persona in esame. **NN_BUDGET** rappresenta il numero massimo di descrittori da prendere in considerazione partendo dall'ultimo. È stato scelto un valore di **50** per ridurre la complessità computazionale e per rimuovere eventuali features "sporche" dopo un periodo di tempo pari a 10 secondi. L'**embedder** è stato accuratamente selezionato, utilizzando il modello ***osnet_x1_0*** addestrato sui dataset specifici per la re-identificazione di esseri umani, ovvero i dataset **Market1501 e DukeMTMC-reID**. Questo modello presenta circa 2.2 milioni di parametri ed è addestrato su immagini di dimensione 256x128.

(Model Zoo, s.d.)

# 3. PAR

In questo capitolo viene descritta la struttura del modello utilizzato per il PAR, con un'analisi dei suoi componenti e delle relative funzioni principali. Vengono inoltre esaminate le caratteristiche del dataset utilizzato per il training e il validation del modello proposto, insieme alle varie tecniche di addestramento implementate, tutte adeguatamente motivate. Infine, viene fornita un'analisi delle performance ottenute durante la fase di addestramento, con una interpretazione appropriata dei risultati.

## 3.1. DATASET

Il dataset utilizzato per addestrare il modello del PAR proviene dal Contest *"PAR Contest 2023: Pedestrian Attributes Recognition with Multi-Task Learning ",* e contiene un totale di 105243 immagini. Queste immagini sono divise in due sottoinsiemi principali: un training set composto da 93081 immagini e un validation set costituito da 12162 immagini.

Ogni immagine nel dataset è annotata con sei attributi distinti:

- Il nome dell’immagine
- Il colore dell'abbigliamento superiore della persona, rappresentato da un valore numerico compreso tra 1 e 11.
- Il colore dell'abbigliamento inferiore della persona, anch'esso rappresentato da un valore numerico compreso tra 1 e 11.
- Il genere della persona, codificato come 0 per maschio e 1 per femmina.
- La presenza di una borsa, indicata come 0 se assente e 1 se presente.
- La presenza di un cappello, segnalata come 0 se assente e 1 se presente.

Il dataset è composto da immagini contenenti solo patch di persone, isolate dal contesto circostante. Tali immagini sono state acquisite da diverse angolazioni e in varie condizioni di illuminazione.

Un aspetto fondamentale del dataset è rappresentato dallo sbilanciamento delle classi. Un gran numero di campioni risulta non etichettato (ciò è indicato con "-1" per l'attributo corrispondente), e si osserva un evidente squilibrio tra le varie categorie. Dai dati riportati, infatti, emerge che per le classi relative al colore dell’abbigliamento, il colore nero prevale sia per la parte superiore che per quella inferiore. Al contrario, colori come l'arancione o il rosa sono sottorappresentati, con pochi esempi associati. Un pattern simile può essere osservato anche per le altre classi:

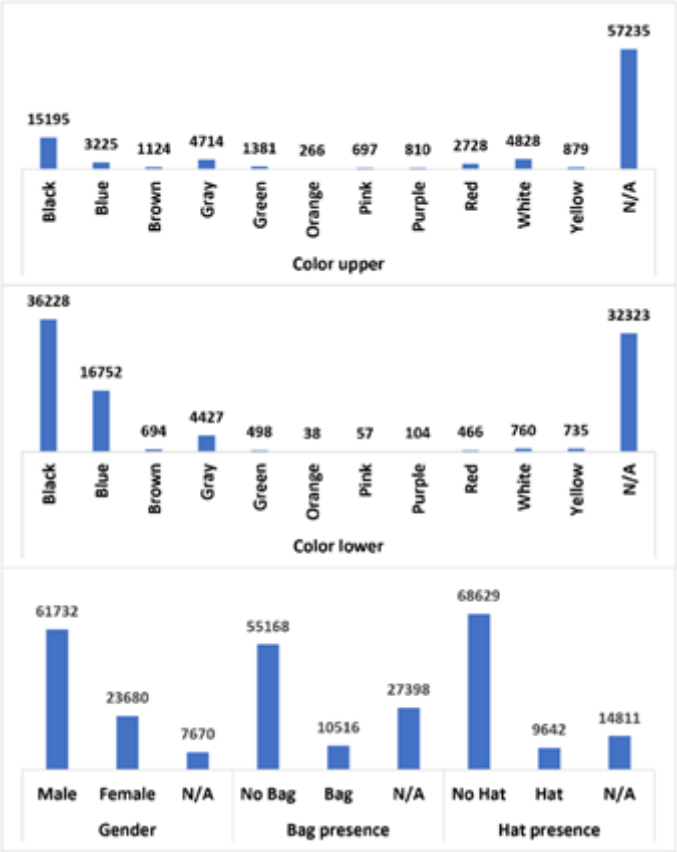


*Figura 4 Rappresentazione del Dataset*

A partire dal dataset fornito, è stata implementata una struttura per contenere le informazioni relative alle etichette delle immagini. In particolare, è stata apportata una modifica agli attributi relativi al colore dell'abbigliamento. Tali valori sono stati decrementati di 1 in modo da poter essere rappresentati in un intervallo compreso tra 0 e 10. Tuttavia, è importante notare che le etichette non valide sono state mantenute con il valore -1.

Ciò è stato fatto per allineare la rappresentazione degli attributi alla convenzione più comune, in cui gli indici iniziano da 0 anziché da 1. Questo ha reso la gestione delle etichette più coerente e ha semplificato l'applicazione delle funzioni di loss che, di default, prendono in input gli indici delle classi, ora compresi tra 0 e 10. In questo modo, la manipolazione dei dati risulta più intuitiva e uniforme durante il processo di addestramento del modello.

## 3.2. NETWORK ARCHITECTURE

Il modello realizzato per il *Pedestrian Attribute Recognition* adotta un approccio di apprendimento **multi-task**. Esso è costituito da una "backbone" impiegata per l'estrazione delle features globali delle immagini, affiancata da cinque "rami", ognuno dei quali è dedicato al riconoscimento di un particolare attributo.

Ciascun ramo è formato da due componenti principali: un modulo di attenzione e un modulo di classificazione. Il modulo di attenzione si focalizza sull'estrazione delle features in una regione specifica dell'immagine. Il modulo di classificazione riceve in input le features estratte dal modulo di attenzione; quindi, valuta e predice l'attributo corrispondente.

*ResNet50* è la **backbone** impiegata nel modello del PAR.

La scelta è stata guidata da diversi motivi.

Rispetto ad architetture più complesse come *ResNet101 o ResNet152, ResNet50* presenta una profondità moderata. Questo equilibrio di profondità permette di mantenere sotto controllo la complessità computazionale, garantendo comunque ottime prestazioni nell'ambito dell'image recognition.

Inoltre, utilizzando un approccio di apprendimento residuo profondo e le skip connections, la rete consente una convergenza più rapida durante l'addestramento.

*ResNet50* è stata pre-addestrata su grandi dataset come *ImageNet*, contenente milioni di immagini appartenenti a migliaia di categorie diverse. Questo pre-addestramento consente al modello di catturare caratteristiche generiche delle immagini, che possono poi essere adattate per risolvere problemi specifici, come quello in questione.

Diverse ricerche hanno evidenziato che *ResNet50* ottiene risultati migliori in termini di accuracy, precision, recall e F1-score rispetto ad altri modelli di base nel campo dell'image recognition. La sua capacità di apprendere e riconoscere dettagli elaborati nelle immagini ha contribuito significativamente al suo successo.

"Image Recognition using ResNet50", (Image Recogniscion using ResNet50, s.d.)

Il **modulo di attenzione** è un componente utilizzato nelle reti neurali per l'elaborazione delle immagini. È progettato per concentrare l'attenzione su determinate regioni di un'immagine durante il processo di apprendimento automatico.

Il modello realizzato per il PAR prevede l'utilizzo del modulo di attenzione CBAM (Content-Based Attention Module).

Tale modulo è composto da due componenti principali: l'attenzione spaziale (Spatial Attention) e l'attenzione di canale (Channel Attention).

L'attenzione di canale si concentra sulle diverse caratteristiche rappresentate da ciascun canale dell'immagine. In un'immagine a colori, infatti, ogni canale rappresenta un aspetto diverso dell'immagine. Questo componente modella le relazioni tra i canali delle feature map, permettendo così di concentrarsi soltanto sui canali più informativi. Tale processo coinvolge l'utilizzo di un'operazione di pooling adattativo, seguita da convoluzioni per catturare le relazioni tra i canali, e infine, una funzione di attivazione *sigmoide* per pesare l'importanza di ciascun canale.

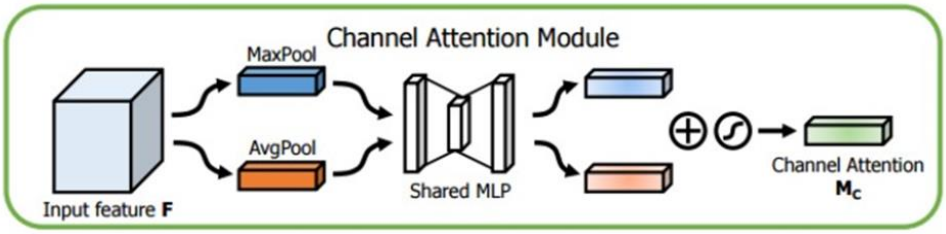


*Figura 5 Modulo di attenzione di canale*

L'attenzione spaziale si concentra sulla localizzazione delle caratteristiche rilevanti all'interno dell'immagine. Essa, quindi, identifica le regioni spaziali più rilevanti all'interno delle feature map. Questa componente utilizza un'operazione di pooling adattativo seguita da convoluzioni per catturare le relazioni spaziali e, successivamente, una funzione di attivazione *sigmoide* per pesare l'importanza delle diverse regioni spaziali.

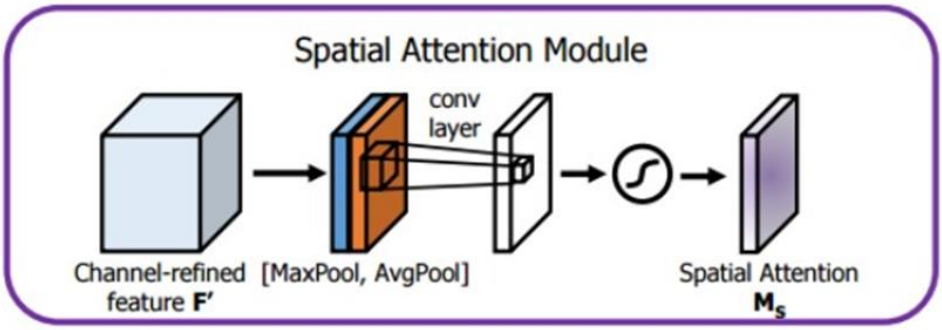


*Figura 6 Modulo attenzione spaziale*

Nell'implementazione del modulo CBAM, l'attenzione di canale e l'attenzione spaziale vengono applicate in parallelo alle features map. Successivamente, i risultati ottenuti vengono combinati tra loro, prima di essere moltiplicati per le features map originali. Questo processo di combinazione consente al modello di trarre vantaggio sia dalle informazioni sui canali che dalle informazioni spaziali, migliorando così la sua capacità di discriminare e focalizzarsi sulle caratteristiche salienti dell'immagine. (GitHub, s.d.)

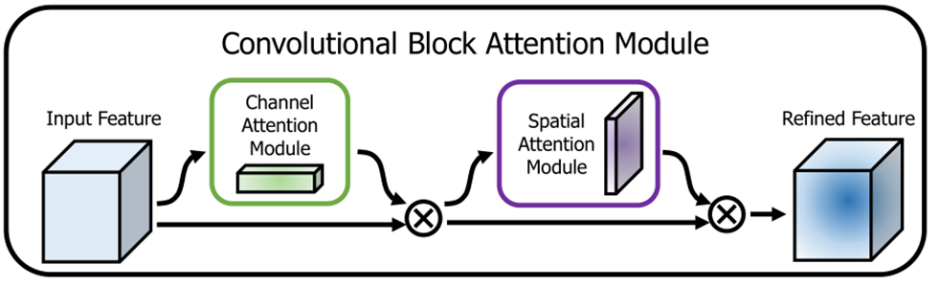


*Figura 7 CBAM*

Il **modulo di classificazione**, basandosi sui risultati forniti dal modulo di attenzione, è responsabile del calcolo delle predizioni relative a ciascun attributo. Nel contesto del PAR, il modello sviluppato impiega tre classificatori binari e due classificatori multi-classe.

- **Classificatori Binari**: Sono caratterizzati da un'unica uscita e vengono utilizzati per la classificazione di attributi il cui valore è pari a 0 e 1. Nell'ambito del PAR, i classificatori binari sono impiegati per rilevare il genere di una persona, la presenza della borsa e la presenza del cappello.
- **Classificatori Multi-classe**: Sono caratterizzati da più di due uscite e vengono utilizzati per la classificazione di attributi aventi più di due possibili valori. Nell'ambito del modello in questione, i classificatori multi-classe vengono impiegati per gestire gli attributi relativi al colore della parte superiore e inferiore dell'abbigliamento. Ogni modulo di classificazione multi-classe è composto da 11 uscite, ognuna corrispondente a un colore.

Entrambe le tipologie di classificatori sono costituite dai seguenti componenti:

- **Layer lineare**: riceve le features estratte dal modulo precedente e le proietta in uno spazio di dimensioni adatte per la classificazione;
- **Funzione di attivazione ReLU**: consente di imparare relazioni complesse non lineari tra le features;
- **Batch Normalization**: normalizza l'output del layer precedente, accelerando la convergenza del modello durante l'addestramento. Inoltre, aiuta a regolarizzare il modello, riducendo il rischio di overfitting;

- **Dropout:** è una tecnica di regolarizzazione che durante l'addestramento "disattiva" casualmente i neuroni del layer, impedendo al modello di dipendere troppo da un particolare insieme di features. Ciò aiuta a migliorare la generalizzazione del modello, rendendolo più robusto e riducendo il rischio di overfitting;
- **Layer lineare**: trasforma le features in vettori di output denominati “logit”. Questi rappresentano i valori di output del secondo layer lineare prima di essere sottoposti a qualsiasi funzione di attivazione.

Nel caso di un classificatore binario, dopo il secondo layer lineare viene applicata una funzione di attivazione *sigmoide*. Questa funzione trasforma l'output del layer in un valore compreso tra 0 e 1, che può essere interpretato come la probabilità che l'input appartenga alla classe 0 o alla classe 1.

D'altra parte, per un classificatore multi-classe, non è necessario applicare alcuna funzione di attivazione dopo il secondo layer lineare. Questo perché si è scelto di utilizzare la funzione Cross-Entropy come funzione di loss durante l'addestramento. La Cross-Entropy include implicitamente l'applicazione della funzione di attivazione *softmax* durante il suo calcolo. Questa funzione di attivazione converte i logit in probabilità su tutte le classi, la cui somma è unitaria; queste probabilità vengono utilizzate per calcolare l'errore tra le predizioni del modello e le etichette di classe reali.

## 3.3. TRAINING

Di seguito si riporta l’approccio utilizzato durante la fase di addestramento.

### 3.3.1 PRE-PROCESSING E DATA AUGMENTATION

Le immagini presenti nel dataset sono state sottoposte a diverse trasformazioni per prepararle all'addestramento del modello. Queste trasformazioni sono state definite tramite il modulo transforms di PyTorch e sono state organizzate in due categorie: una per il set di addestramento e una per il set di validazione.

Per il set di addestramento, le trasformazioni impiegate sono:

- **transforms.Resize((224, 224))**: poiché la backbone utilizzata prevede una dimensione delle immagini di input pari a 224x224, ciascuna immagine del dataset deve essere ridimensionata a queste stesse dimensioni.
- **transforms.RandomAffine(degrees=30, shear=10)**: viene applicata una trasformazione geometrica casuale, con una rotazione massima di 30 gradi e una deformazione massima di 10 gradi. Questa trasformazione aiuta a introdurre variazioni e robustezza nel modello.
- **transforms.RandomHorizontalFlip()**: le immagini vengono ribaltate orizzontalmente in modo casuale con una probabilità del 50%. Questo aiuta ad aumentare la variabilità dei dati e a migliorare la capacità del modello di generalizzare su diverse orientazioni delle immagini.
- **transforms.ToTensor()**: le immagini vengono convertite in tensori PyTorch.

- **transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])**: le intensità dei pixel delle immagini vengono normalizzate utilizzando i valori di media e deviazione standard comunemente utilizzati nel dataset ImageNet. Questa normalizzazione aiuta a garantire che i valori dei pixel abbiano una scala appropriata per il modello durante l'addestramento.

Per le immagini del set di validazione sono state applicate le stesse trasformazioni ad esclusione di quelle geometriche casuali, in modo che il modello possa essere valutato su immagini non modificate. Sono state escluse trasformazioni sui colori per evitare che la rete potesse apprendere informazioni errate riguardo i colori riducendone le performance.

### 3.3.2 CUSTOM SAMPLER

Poiché il training set a disposizione contiene numerosi campioni non etichettati, è stata implementata in Pytorch la classe *CustomRandomSampler()* per la creazione personalizzata dei batch durante la procedura di addestramento.

Quando viene creato un batch, si verifica se in esso è presente almeno un campione con un'etichetta valida per ciascun attributo. Se ciò non accade, lo si sostituisce con un altro campione casuale dal dataset, che ha un'etichetta valida per quell'attributo, consentendo eventualmente l'introduzione di campioni già impiegati in batch precedenti.

Questo approccio garantisce che ogni batch contenga un insieme di etichette valide per l'addestramento del modello, evitando che la loss possa restituire valori non veritieri riguardo l'addestramento dei singoli task: potrebbe capitare che in uno o più batch non ci siano campioni validi per cui la funzione di loss non verrebbe calcolata, indicando erroneamente un errore nullo; dunque, ciò garantisce che la loss sia calcolata per tutti i task per ogni batch.

### 3.3.3 PROCEDURA DI ADDESTRAMENTO

Per la fase di addestramento del modello, al fine di applicare la tecnica del transfer learning, sono stati utilizzati i pesi pre-addestrati di ResNet50 sul dataset ImageNet. L'ultimo layer *fully connected* del modello è stato sostituito con i moduli di attenzione e di classificazione già presentati in precedenza all'interno di questo capitolo.

Sono state eseguite due fasi di addestramento, ognuna con una diversa impostazione dei parametri sulla rete ResNet50.

In una **prima fase,** il modello è stato addestrato mantenendo **congelata la backbone** impiegata per la features extraction, così da conservare la rappresentazione appresa durante il pre-addestramento. Questa fase ha visto l'esecuzione di **13 epoche** di addestramento, con ottimizzatore **Adam** e **learning rate** pari a **0,001**.

La **seconda fase** ha riguardato l'esecuzione del fine-tuning. I layer superiori della backbone sono stati scongelati e addestrati insieme ai layer dei moduli di attenzione e di classificazione, mentre i rimanenti layer inferiori non sono stati sbloccati. Ciò è stato fatto perché i layer inferiori della rete estraggono caratteristiche più generali; i layer superiori, invece, devono

essere riaddestrati al fine di rendere l'estrazione delle features più rilevanti per il compito specifico. In particolare, sono stati eseguiti **22 epoche** di addestramento **sbloccando gli utlimi 5 layer della backbone** con ottimizzatore **Adam** e **learning rate** pari a **0,0001.**

Durante gli addestramenti è stato utilizzata la tecnica dell'early stopping con *PATIENCE* pari a 5.

### 3.3.4. CUSTOM LOSS

Per il calcolo della loss durante l'addestramento, è stata utilizzata la Binary Cross Entropy per i classificatori binari e la Cross Entropy per i classificatori multi-classe. Inoltre, è stato necessario implementare una maschera per gestire i campioni non annotati nel dataset.

La maschera viene utilizzata per escludere tali campioni dal calcolo della loss relativa a uno specifico task durante l'addestramento del modello. Una maschera è una matrice booleana che ha la stessa dimensione dei tensori delle etichette e dei valori di output del modello. Se un campione ha un'etichetta valida per quell'attributo, il valore corrispondente nella maschera è True; altrimenti, è False. Durante il calcolo della loss, i campioni contrassegnati come False nella maschera vengono ignorati.

Per la Binary Cross Entropy, è stata necessaria un'implementazione esplicita della maschera, mentre per la Cross Entropy questo comportamento è già incorporato nella funzione di loss stessa.

In quest'ultimo caso, infatti, è stato sufficiente passare il valore dell'etichetta da ignorare alla funzione, che automaticamente esclude i campioni corrispondenti dalla computazione della loss.

Inoltre, è stata sviluppata in Pytorch la funzione *make_weights()* per calcolare i pesi associati alle classi dei dati presenti nel dataset. Tali pesi sono stati utilizzati durante il calcolo della funzione di loss per affrontare lo sbilanciamento nella distribuzione delle classi.

La funzione conta il numero di occorrenze valide di ciascun attributo nel dataset. Successivamente, ottiene **i pesi associati a ciascuna classe calcolando l'inverso del numero di occorrenze** della classe stessa. In questo modo, si garantisce che le classi più rare abbiano un peso maggiore durante l'addestramento del modello. Ciò contribuisce a bilanciare l'impatto delle diverse classi durante la fase di apprendimento, migliorando così le prestazioni del modello su dati sbilanciati.

Ad ogni epoca di addestramento del modello, dopo aver calcolato le loss per ciascun attributo, queste vengono sommate per ottenere una loss aggregata. Quest'ultima rappresenta l'errore totale del modello rispetto a tutte le classi. La loss aggregata viene poi utilizzata per calcolare i gradienti e aggiornare i pesi del modello durante la fase di backward.

Si noti che per i problemi di classificazione binaria è stata fornita anche l'implementazione di una **Binary Asymmetric Loss**, appositamente progettata per gestire lo sbilanciamento del dataset.

Una generica forma per questa funzione di loss è data dalla seguente formula:

$$L = -yL_+ - (1-y)L_-$$

in cui con $y$ si indicano le etichette reali dei campioni; $L_-$ ed $L_+$ sono rispettivamente la componente di loss relativa ai campioni negativi (classe 0) e quella relativa ai campioni positivi (classe 1).

Si definiscono $L_+$ e $L_-$ come segue:

$$\begin{cases} L_+ = (1-p)^{\gamma_+} \log(p) \\ \\ L_- = p^{\gamma_-} \log(1-p) \end{cases}$$

*p* sono le predizioni restituite dal modello; $\gamma^+$ e $\gamma^-$ rappresentano i parametri di focusing che regolano il contributo dato dai campioni positivi e dai campioni negativi. Se l'obiettivo è enfatizzare il contributo dei campioni positivi, allora si definisce $\gamma^- > \gamma^+$.

In linea generale, l'asymmetric focusing è una tecnica utilizzata per gestire in modo appropriato i contributi dei campioni positivi e negativi nella loss, consentendo al modello di imparare caratteristiche significative anche dai campioni appartenenti alla classe meno frequente. Nell'ambito del Pedestrian Attribute Recognition, i parametri $\gamma^-$ e $\gamma^+$ sono stati impostati utilizzando i pesi associati alle classi tramite la funzione *make_weights()*, introdotta precedentemente. (Fonte: (Corneal University, s.d.) )

Dopo una serie di test, è stato notato che i risultati dei modelli addestrati con la binary asymmetric loss su nuovi campioni non rispecchiavano adeguatamente la realtà. Di conseguenza, è stata presa la decisione di utilizzare la binary cross entropy loss.

### 3.3.5. EVALUATION PROTOCOL

Il modello presentato in questo documento viene valutato in termini di accuracy.

Come riportato dagli organizzatori del contest, l'accuracy è data dal rapporto tra il numero di predizioni corrette e il numero totale di predizioni effettuate:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{All predictions}}$$

Poiché ci sono 5 task differenti da classificare, in ogni epoca del processo di addestramento sono state calcolate 5 accuracy, una per ogni task. Al termine dell'epoca, tali accuracy sono state sommate e successivamente divise per il numero di task.

### 3.4. EXPERIMENTAL RESULTS

Analizzando i risultati ottenuti dalle diverse fasi di addestramento del modello di Pedestrian Attribute Recognition, è possibile osservare **una loss pari a 2.6 e un'accuracy del 98% sul validation, mentre sul training abbiamo una loss di 2.2 e un'accuracy del 0.94%.** Una loss più alta sul validation rispetto al training suggerisce un modello in overfitting. Invece, un'accuracy maggiore sul validation anziché nel training indica l'esistenza di un validation poco rappresentativo rispetto alle caratteristiche più complesse che il modello apprende durante la fase di training.

Tuttavia, i risultati ottenuti sono principalmente correlati allo sbilanciamento delle classi presenti nel dataset. Nonostante sia stata implementata una pesatura intra-classe, tale strategia non è stata estesa alla loss aggregata, la quale si limita a computare la somma delle singole loss di ciascun task. Poiché questa somma non è pesata, ciascuna loss viene considerata allo stesso modo delle altre, portando automaticamente a un apprendimento sbilanciato tra i task. Una possibile soluzione sarebbe quella di introdurre una somma pesata nel calcolo della loss aggregata, i cui pesi sono calcolati in funzione del numero di occorrenze di campioni validi per ciascun task.

Infine, il modello è stato sottoposto a un rigoroso processo di test utilizzando una varietà di video registrati in diverse condizioni di illuminazione (sia artificiale che naturale) e in diversi momenti della giornata. Questo approccio ha permesso di esaminare le prestazioni del modello sotto varie condizioni, inclusi scenari affollati e situazioni con poche persone. L'obiettivo è stato individuare i punti di forza e di debolezza del modello addestrato, al fine di identificare le limitazioni e apportare eventuali correzioni. Tutti i video utilizzati per il test sono stati registrati nell'atrio Cues.

## 4. APPLICATION

Nel presente capitolo, sarà esaminata l'interazione tra le componenti delineate nei capitoli precedenti al fine di realizzare il risultato finale, come definito nel contest. Sarà posto un particolare focus sull'output visivo, sulle operazioni di classificazione eseguite dal modello e sulla creazione dei risultati nel formato JSON di output.

### 4.1. UPDATE ROI

La funzione *update_roi* svolge un'analisi spaziale e temporale delle persone tracciate all'interno delle regioni di interesse (ROI). Le regioni d'interesse possono essere configurate mediante un file di configurazione denominato *config.txt*. Di seguito ne è riportato un esempio:

```
{
    "roi1":{
        "x":0.1,
        "y":0.2,
        "width":0.4,
        "height":0.4
    },
    "roi2":{
        "x":0.5,
        "y":0.7,
        "width":0.5,
        "height":0.3
    }
}
```

*Figura 8 Configurazione ROI*

Il metodo preposto alla configurazione delle ROI *è read_roi* ed è situato nella classe *FileJson*. La funzione apre il file JSON in modalità lettura e restituisce due dizionari contenti le informazioni sulle regioni di interesse che sono punto in alto a sinistra della ROI, la larghezza e l'altezza dei bounding box che le rappresenta. Questi valori vengono poi moltiplicati per la dimensione del frame di elaborazione con lo scopo di ottenere delle misure assolute.

Una volta che le ROI sono state definite, la funzione calcola il centro del bounding box di ciascuna persona e verifica se si trova all'interno della ROI1 o ROI2. Ad ogni nuovo ingresso di una persona, vengono impostati dei flag booleani su True e il contatore di attraversamento viene incrementato. Ad ogni nuova uscita, il flag viene reimpostato su False. Quando una persona si trova all'interno di una delle regioni di interesse, viene incrementato il tempo di permanenza di un valore costante denominato **SAMPLE_TIME**, ottenuto come l'inverso degli fps considerati per l'elaborazione del video.

## 4.2. UPDATE PAR

La funzione *update_par* classifica le persone all'interno della scena utilizzando il modello *par_model* salvando i risultati negli attributi dell'oggetto *CustomTrack*. Per ogni frame vengono eseguiti i seguenti passaggi:

1. Estrae l'immagine della persona dal frame utilizzando il bounding box dell'oggetto tracciato tramite la finzione *_crop_image*.
2. Applica il modello *par_model* alla patch ottenendo una lista di predizioni per ogni compito di classificazione. Nel caso di task multiclasse viene utilizzata la funzione di attivazione *softmax* e successivamente la funzione di *argmax* per ottenere l'indice della classe più probabile. Se il task è binario si verifica che la probabilità derivante dalla *sigmoide* sia maggiore di una soglia pari a 0.5
3. La predizione viene aggiunta al dizionario delle misurazioni dello specifico track utilizzando il metodo *add_par_measurement*

Dato che l'analisi può avvenire su più frame, per ogni persona tracciata vengono raccolte un totale di 25 misurazioni per ogni attributo. Una volta completate le misurazioni, il track viene contrassegnato come classificato e quindi popola i propri attributi con le predizioni più frequenti attraverso il metodo *find_max*. Questo approccio consente di rendere le misurazioni più robuste, ad esempio in presenza di possibili occlusioni o cambi di illuminazione. Alla fine dell'esecuzione dell'applicazione, tutti i track osservati nella scena vengono scritti su file *results.txt* utilizzando la funzione *write_par* della classe *FileJson*, in ordine crescente di *ID*. Un esempio di output è il seguente:

```
1. {
2.     “people” : [
3.         {
4.             “id”: 1,
5.             “gender” : “male”,
6.             “bag” : true,
7.             “hat” : false,
8.             “upper_color” : “red”,
9.             “lower_color” : “blue”,
10.            “roi1_passages” : 1,
11.            “roi1_persistence_time” : 32,
12.            “roi2_passages” : 2,
13.            “roi2_persistence_time” : 45
14.        },
15.        {
16.            “id” : 2,
17.            “gender” : “female”,
18.            “bag” : false,
19.            “hat” : true,
20.            “upper_color” : “black”,
21.            “lower_color” : “white”,
22.            “roi1_passages” : 0,
23.            “roi1_persistence_time” : 0,
24.            “roi2_passages” : 1,
25.            “roi2_persistence_time” : 12
26.        }
27.    ]
28. }
```

*Figura 9 JSON Results*

Per ogni persona tracciata vengono stampate le seguenti informazioni:

- **id**: identificativo univoco. È un numero progressivo che aumenta il suo valore nel tempose passa una persona all'interno della scena e la identifica in maniera univoca.
- **gender**: male o female. Indica il sesso della persona.
- **bag**: true o false. Se settato a True, indica la presenza di un individuo con la borsa, come zaino, ventiquattrore, ecc. Oppure False se senza borsa.
- **hat**: true o false. Se settato a True, indica la presenza di un individuo con cappello, come berretto, cappello, ecc. Oppure False se senza cappello.
- **upper_color**: black, blue, brown, gray, green, orange, pink, purple, red, white o yellow. Rappresenta un indumento per la parte superiore del corpo, come giacca, maglia, cappotto, ecc.
- **lower_color**: black, blue, brown, gray, green, orange, pink, purple, red, white o yellow. Rappresenta un indumento per la parte inferiore del corpo, come pantalone, leggins, jeans, ecc.
- **roi1_passages**: conta il numero di passaggi in ROI1.
- **roi1_persistence_time**: tempo totale speso in ROI1, espresso in secondi.
- **roi2_passages**: conta il numero di passaggi in ROI2.
- **roi2_persistence_time**: tempo totale speso in ROI2, espresso in secondi.

## 4.3. PRINT_SCENE

L'ultima funzione è la *print_scene*, che visualizza l'output della scena secondo le direttive del contest. Essa mostra i bounding box delle persone con colori diversi a seconda della loro posizione rispetto alle due regioni di interesse (roi1 e roi2), che sono disegnate sul frame con la funzione *print_roi*. Questa funzione usa le coordinate delle roi dal file JSON fornito dalle specifiche del contest.

I bounding box sono blu se le persone sono in roi1, verdi se sono in roi2 e rossi se non sono in nessuna roi. Anche l'id delle persone cambia colore in base alla loro posizione. Inoltre, nell'angolo in alto a sinistra del video si visualizzano il numero totale corrente di persone nella scena, il numero corrente di persone nelle roi e il numero totale di passaggi per ogni roi. Vengono riportante anche le informazioni di ogni persona nella scena, come genere, presenza di borsa e/o cappello e colore della parte superiore e inferiore dei vestiti indossati.

# Bibliografia

(s.d.). Tratto da https://arxiv.org/abs/1602.00763

(s.d.). Tratto da https://www.researchgate.net/publication/358134782_Sort_and_Deep-SORT_Based_Multi-Object_Tracking_for_Mobile_Robotics_Evaluation_with_New_Data_Association_Metrics

*Corneal University*. (s.d.). Tratto da https://arxiv.org/pdf/2009.14119.pdf

*GitHub*. (s.d.). Tratto da https://github.com/luuuyi/CBAM.PyTorch

*Image Recogniscion using ResNet50*. (s.d.). Tratto da https://www.eurchembull.com/uploads/paper/3d3038bc3e7d1fad63f18fa13fe711ac.pdf

*Model Zoo*. (s.d.). Tratto da Torchreid: https://kaiyangzhou.github.io/deep-person-reid/MODEL_ZOO.html

*ResNet50*. (s.d.). Tratto da https://medium.com/@nitishkundu1993/exploring-resnet50-an-in-depth-look-at-the-model-architecture-and-code-implementation-d8d8fa67e46f

Ricardo Pereira, G. C. (s.d.). *ResearchGate*. Tratto da https://www.researchgate.net/publication/358134782_Sort_and_Deep-SORT_Based_Multi-Object_Tracking_for_Mobile_Robotics_Evaluation_with_New_Data_Association_Metrics

*Sort and Deep-SORT Based Multi-Object Tracking for Mobile Robotics: Evaluation with New Data Association Metrics*. (s.d.).

*ultralytics*. (s.d.). Tratto da https://docs.ultralytics.com/it/models/yolov8/

University, C. (s.d.). Tratto da https://arxiv.org/abs/1703.07402

University, C. (s.d.). *arXiv*. Tratto da https://arxiv.org/abs/1602.00763

University, C. (s.d.). *arXiv*. Tratto da https://arxiv.org/abs/1703.07402

# Indice delle Figure

Figura 1 Analisi delle prestazioni Yolo 8 ..... 4
Figura 2 Analisi velocità di processamento Yolo 8 ..... 4
Figura 3 Analisi Performance Deep Sort ..... 7
Figura 4 Rappresentazione del Dataset ..... 9
Figura 5 Architettura del modello ResNet50 ..... **Errore. Il segnalibro non è definito.**
Figura 6 Modulo di attenzione di canale ..... 11
Figura 7 Modulo attenzione spaziale ..... 11
Figura 8 CBAM ..... 12
Figura 9 Configurazione ROI ..... 18
Figura 10 JSON Results ..... 19